



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A perfect table-driven agent would need to store an action for every possible percept sequence. Lecture 02 defines the agent function as a mapping from all percept histories to actions, written as:

$$f: P^* \rightarrow A$$

As the agent's lifetime increases, the number and length of possible percept sequences increase enormously. The table would need to contain an entry for every possible history, causing a combinatorial explosion. For complex environments such as Chess, the number of possible histories is so large that the table cannot realistically be stored or programmed. Therefore, a practical agent program must compute actions using algorithms and memory rather than store every possible answer.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

In the code:

```
if percept["wall_ahead"]:
    return "Left"
```

This represents if wall_head is true, Then choose the action as left.

In the code:

```
if percept["food_here"]:
    return "Up"
```

This maps the current sensor condition food_here to a fixed action.

These are condition-action rules because each current percept condition is directly mapped to an action. The simple reflex agent does not use percept history or internal memory.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The environment became partially observable because the agent received only:

"wall_ahead": True/False,

"food_here": True/False

It did not know its exact position, the full wall map, where it had already moved, what action it previously attempted, Lecture 02 explains that partial observability means the sensors do not reveal the complete state of the environment.

The simple reflex agent also does not store the percept sequence. Therefore, when it receives the same local percept again, it cannot distinguish whether it is in a new place or repeating an old situation. The same condition triggers the same action repeatedly, which can create an infinite loop around a wall or corner.

Lecture 02 defines the percept sequence as the full history of sensory inputs. Because the simple reflex agent ignores that history, it cannot use past failures to improve the next decision.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In my ModelBasedAgent, the internal state is represented by variables such as:

```
self.last_percept  
self.last_action  
self.failed_actions
```

Before choosing a new action, the agent compares the current percept with its previous action and stored memory.

```
if self.last_action is not None and percept["wall_ahead"]:  
    self.failed_actions.add(self.last_action)
```

The Transition Model represents how the world evolves. The current wall_ahead and food_here readings inform the agent about the latest state of the environment. By comparing these readings with self.last_percept, the agent can recognize changes in the world, such as encountering a wall after previously observing a clear path.

The Sensor Model, according to the terminology used in the lecture, represents how the agent's actions affect the world. By storing self.last_action, the agent can associate an action with its observed result. For example, if the agent previously selected "Up" and now detects a wall, it records "Up" as an unsuccessful action in self.failed_actions.

After updating this internal state, the agent checks its memory before applying its condition-action rules. It avoids actions recorded as unsuccessful and selects a different direction. Therefore, unlike a simple reflex agent, it does not react only to the current percept, it combines the current sensor reading with previous percepts and actions to build a small internal model and reduce repeated loops.



Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING